Introdução à Orientação a Objetos

Aula 05 — Arrays (arranjos)

Programação Orientada a Objetos

Semana 5

Sumário

1. Introdução
2. O que é um array
3. O array no CarrinhoDeCompras
4. Percorrendo o array
5. Inicialização e arrays de primitivos
6. Onde o array trava
7. Exercício de fixação
8. Encerramento

Revisão da aula anterior

- Construtor: mesmo nome da classe, sem retorno, executado no `new`
- Construtor com parâmetro **obriga** dependências na criação do objeto
- Sobrecarga: mesmo nome, listas de parâmetros diferentes
- `this()` delega para outro construtor, sempre na primeira linha

Nosso projeto: `Livro`, `Autor`, `Produto` e `CadastroDeLivros` já encapsulados e com construtores.

Uma nova classe: o carrinho de compras

O cliente da livraria vai comprar vários livros de uma vez. Precisamos guardar todos eles.

```
1 public class CarrinhoDeCompras {
2     private double total;
3     private Livro livro1;
4     private Livro livro2;
5     private Livro livro3;
6     private Livro livro4;
7 }
```

- Quantos livros cabem nesse carrinho?
- E se o cliente comprar uma coleção inteira?

O método adiciona vira uma escada

```
1  public void adiciona(Livro livro) {
2      if (this.livro1 == null) {
3          this.livro1 = livro;
4      } else if (this.livro2 == null) {
5          this.livro2 = livro;
6      } else if (this.livro3 == null) {
7          this.livro3 = livro;
8      } else if (this.livro4 == null) {
9          this.livro4 = livro;
10     } else {
11         System.out.println("Nao tem mais espaco");
12         return;
13     }
14     this.total += livro.getValor();
15 }
```

E para mostrar os livros do carrinho?

- Seriam **os mesmos ifs** outra vez, agora para imprimir
- E de novo para calcular, de novo para remover, de novo para procurar...
- Cada livro a mais no carrinho = mais um atributo e mais um `if` em cada método

Esse código é quase impossível de manter.

Uma dívida da aula 01

```
1 public static void main(String[] args) {
2 }
```

- Você digita isso desde a primeira aula
- Na aula 01 dissemos apenas: "`args` guarda os argumentos da linha de comando"
- E ficou faltando explicar aqueles colchetes

`String[]` é um **array**. Hoje a dívida é paga.

Hoje

- O que é um **array** e como criá-lo
- Guardando muitos objetos em um único atributo
- Percorrendo: `for` com índice e *enhanced-for*
- `length`, índices e o famoso `ArrayIndexOutOfBoundsException`
- Onde o array não ajuda — e o que vem depois

Declarando um array

Basta acrescentar `[]` depois do tipo:

```
1 Livro livro;      // uma referencia para um livro
2 Livro[] livros;   // uma referencia para varios livros
```

- `Livro[]` é lido como "array de `Livro`"
- Funciona com qualquer tipo: `int[]`, `double[]`, `String[]`, `Produto[]`
- Um array guarda elementos de **um único tipo**

Um array é um objeto

Declarar não basta — como todo objeto, o array precisa ser criado com `new`:

```
1 Livro[] livros = new Livro[10];
```

- Repare na sintaxe: o tamanho vai entre os colchetes
- Criamos aqui um array de **10 posições**
- Antes do `new`, a variável `livros` vale `null` — como qualquer referência

O tamanho é obrigatório e definitivo

```
1 Livro[] livros = new Livro[];     // NAO COMPILA: falta o tamanho
2 Livro[] livros = new Livro[10];   // ok
```

- Ao criar um array, você é **obrigado** a informar quantas posições ele terá
- E esse tamanho **não muda** depois. Nunca.
- Encheu as 10 posições? Não existe "adicionar a 11ª"

Guarde essa limitação — voltaremos a ela no fim da aula.

Os índices começam em zero



- Os índices vão de 0 até **tamanho − 1**
- Um array de 10 posições vai de 0 a 9
- **A posição 1 é a segunda posição**, não a primeira
- Não existe a posição 10 nesse array — e tentar acessá-la quebra o programa

As posições nascem vazias

Ao criar o array, todas as posições recebem o valor padrão do tipo:

Tipo do array	Cada posição começa com
`int[]`, `long[]`	`0`
`double[]`	`0.0`
`boolean[]`	`false`
`Livro[]`, `String[]`	`null`

- É a mesma tabela de valores padrão da aula 04
- `new Livro[10]` cria o array, **mas não cria nenhum livro**

Lendo e escrevendo posições

```
1 Livro[] livros = new Livro[10];
2
3 livros[0] = new Livro(autor);   // escreve na primeira posicao
4 livros[1] = new Livro(autor);   // escreve na segunda
5
6 System.out.println(livros[0].getValor());  // le a primeira
7 System.out.println(livros[5]);             // null: ninguem escreveu ainda
```

- Mesma sintaxe dos dois lados: `array[indice]`
- Cuidado ao ler uma posição `null` e chamar um método nela

Quatro atributos viram um

```java
public class CarrinhoDeCompras {
    private double total;
    private Livro[] livros = new Livro[10];

    public void adiciona(Livro livro) {
        // precisamos adicionar no array
    }
}
```

- Sumiram `livro1`, `livro2`, `livro3` e `livro4`
- O carrinho está limitado a 10 livros — uma decisão nossa, não uma imposição

Primeira tentativa

```
1 public void adiciona(Livro livro) {
2     System.out.println("Adicionando: " + livro);
3     this.livros[1] = livro;
4     this.total += livro.getValor();
5 }
```

- Compila, executa e o total sai certo
- Mas **todos** os livros vão para a mesma posição
- Cada um sobrescreve o anterior: no fim, o carrinho tem **um** livro

E se quem chama informar a posição?

```java
public void adiciona(Livro livro, int indice) {
    this.livros[indice] = livro;
    this.total += livro.getValor();
}
```

```java
carrinho.adiciona(livro1, 0);
carrinho.adiciona(livro2, 0);   // apagou o livro1!
```

- Deixar esse controle na mão de quem chama o método é arriscado
- É o mesmo raciocínio do encapsulamento: a classe deve cuidar das próprias regras

A solução: um contador

```
1  public class CarrinhoDeCompras {
2      private double total;
3      private Livro[] livros = new Livro[10];
4      private int contador = 0;
5
6      public void adiciona(Livro livro) {
7          System.out.println("Adicionando: " + livro);
8          this.livros[contador] = livro;
9          this.contador++;
10         this.total += livro.getValor();
11     }
12
13     public double getTotal() {
14         return this.total;
15     }
16 }
```

Por que o contador funciona

Chamada	Escreve em	`contador` depois
1ª `adiciona`	`livros[0]`	1
2ª `adiciona`	`livros[1]`	2
3ª `adiciona`	`livros[2]`	3

- O contador guarda **a próxima posição livre**
- Ele começa em 0, que é justamente o primeiro índice válido
- E também nos diz quantos livros já foram adicionados

Ainda deixamos passar uma coisa

- O array tem 10 posições e o contador só cresce
- Na 11ª chamada de `adiciona`, ele tenta escrever em `livros[10]`
- Essa posição **não existe**

O que acontece? Vamos provocar o erro no próximo bloco.

`length`: o tamanho do array

```
1 Livro[] livros = new Livro[10];
2 System.out.println(livros.length);    // 10
```

- `length` devolve o **tamanho** do array, não a quantidade de posições preenchidas
- No nosso carrinho com 3 livros, `livros.length` continua valendo 10
- Quantos livros existem de verdade? Isso quem sabe é o `contador`

Atenção: `length` x `length()`

```
1 livros.length       // ARRAY: atributo, sem parenteses
2 nome.length()       // STRING: metodo, com parenteses
```

	Escrita	**O que é**
Array	`livros.length`	atributo
String	`nome.length()`	método

Trocar um pelo outro é erro de compilação — e é um dos mais comuns da linguagem.

Percorrendo com `for`

```
1 public void mostrarItens() {
2     for (int i = 0; i < this.livros.length; i++) {
3         Livro livro = this.livros[i];
4         if (livro != null) {
5             System.out.println(livro.getValor());
6         }
7     }
8 }
```

- `i` começa em 0, o primeiro índice
- A condição é `i < length`, e não `i <= length`

O erro clássico

```
1 for (int i = 0; i <= this.livros.length; i++) {
2     System.out.println(this.livros[i]);
3 }
```

- Com <=, o `i` chega a 10 em um array de tamanho 10
- Mas os índices válidos vão só até 9
- O programa **compila** — e quebra na execução

Erro de compilação o compilador acha. Esse aqui só aparece rodando.

Lendo a *stacktrace*

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 10 out of bounds for length 10
    at CarrinhoDeCompras.mostrarItens(CarrinhoDeCompras.java:24)
    at TesteCarrinho.main(TesteCarrinho.java:12)
```

- **Primeira linha:** o nome do problema e o índice que estourou (10)
- **Demais linhas:** a classe, o método e a **linha exata** onde ocorreu
- Leia sempre de cima para baixo: o erro está no topo

Parece assustador, mas a *stacktrace* está te entregando o endereço do problema.

O *enhanced-for* (Java 5)

```
1 for (Livro livro : this.livros) {
2     if (livro != null) {
3         System.out.println(livro.getValor());
4     }
5 }
```

Leia como: "**para cada** `Livro` **livro dentro de** `livros`".

- Sem índice, sem `length`, sem `i++`
- Sem chance de escrever <= e estourar o array

Qual dos dois usar?

Preciso do índice	`for` clássico
Preciso escrever nas posições	`for` clássico
Só quero ler os elementos	*enhanced-for*

- O *enhanced-for* é mais curto e mais seguro — prefira quando der
- Ele não serve para trocar o conteúdo das posições do array
- Também funciona com outras estruturas, que veremos na aula 09

Cuidado com as posições vazias

```
1 Livro[] livros = new Livro[10];
2 livros[0] = new Livro(autor);
3
4 for (Livro livro : livros) {
5     System.out.println(livro.getValor());  // quebra na segunda volta
6 }
```

- O array tem 10 posições, mas só 1 livro
- Nas outras 9, `livro` vale `null` — e `null` não tem `getValor()`
- Por isso o `if (livro != null)` aparece em todos os exemplos

Criando o array já preenchido

```java
int[] notas = {7, 8, 10, 6};
String[] nomes = {"Maria", "Joao", "Ana"};
```

- Não escrevemos `new`, nem o tamanho
- O tamanho é deduzido pela quantidade de elementos: `notas.length` vale 4
- Muito útil para valores fixos e conhecidos

Fora da declaração, o new volta

```
1 int[] notas;
2 notas = {7, 8, 10, 6};            // NAO COMPILA
3 notas = new int[]{7, 8, 10, 6}; // ok
```

- A forma curta só vale na mesma linha da declaração
- Nos demais casos, escreva `new int[]{...}` — sem o tamanho entre colchetes

Uma sintaxe alternativa (que você não deve usar)

```
1 int[] notas;    // recomendado: o tipo e "array de int"
2 int notas[];    // tambem compila, herdado do C
```

- As duas formas fazem exatamente a mesma coisa
- A primeira deixa claro que o **tipo** da variável é "array de `int`"
- Mostramos a segunda só para você conseguir ler código dos outros

Array é uma referência

```java
int[] a = {1, 2, 3};
int[] b = a;

b[0] = 99;

System.out.println(a[0]);    // 99 !
```

- A atribuição **não copia** o array
- a e b são dois nomes para o **mesmo** objeto na memória
- Alterar por um nome altera para o outro

Por que isso importa

- É o mesmo comportamento de qualquer objeto, desde a aula 02
- Um `getLivros()` que devolve o array deixa quem chamou mexer no array **de dentro** do carrinho
- Encapsulamento não é só `private`: é também não entregar suas estruturas internas

Um array `private` devolvido por um *getter* não é mais privado.

Arrays de duas dimensões *(leitura complementar)*

```java
int[][] matriz = new int[3][4];    // 3 linhas, 4 colunas

matriz[0][1] = 5;

for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

- Um array de duas dimensões é, na verdade, um **array de arrays**
- Por isso `matriz.length` é o número de linhas e `matriz[i].length`, o de colunas

Conteúdo de leitura: não será cobrado na lista nem na prova.

Limitação 1: o tamanho é fixo

- Nosso carrinho aceita 10 livros. O 11º não entra.
- Aumentar significa criar um array maior e copiar tudo na mão
- Escolher um tamanho grande "por garantia" desperdiça memória

Quantos livros um cliente vai comprar? Ninguém sabe na hora de escrever o código.

Limitação 2: remover deixa buraco

```java
livros[1] = null;   // removi o segundo livro
```

livro A	null	livro C	null

- O array não "fecha" o espaço sozinho
- Ou você convive com buracos, ou move todos os elementos seguintes na mão
- E o `contador` passa a mentir sobre a próxima posição livre

Limitação 3: o controle é todo manual

- Você escreve o `contador`
- Você verifica se ainda cabe
- Você testa `null` em cada volta do laço
- Você move os elementos ao remover

Todo esse controle é código repetido em toda classe que guarda coleções.

E código repetido é lugar onde bug mora.

Então por que aprender array?

- É a estrutura mais básica: tudo o que vem depois é construído sobre ela
- Aparece na API o tempo todo — a começar pelo `String[] args`
- É rápido e direto quando o tamanho é conhecido e fixo

A solução para as três limitações se chama `List`.

Ela chega na aula 09 — e é construída sobre arrays.

Um estoque de produtos

Retomando a `Produto` das aulas anteriores:

```
1 public class Produto {
2     private String nome;
3     private double preco;
4
5     public Produto(String nome, double preco) { ... }
6     public String getNome() { ... }
7     public double getPreco() { ... }
8 }
```

Crie a classe `EstoqueDeProdutos`:

1. um array de **5** produtos, um `contador` e o método `adiciona(Produto)`
2. `getValorTotal()`, somando o preço de todos, com *enhanced-for*
3. `getProdutoMaisCaro()`, devolvendo o produto de maior preço
4. chame `adiciona` **seis** vezes e explique o que acontece

Solução — a classe e o adiciona

```java
public class EstoqueDeProdutos {
    private Produto[] produtos = new Produto[5];
    private int contador = 0;
    public void adiciona(Produto produto) {
        this.produtos[contador] = produto;
        this.contador++;
    }
    public double getValorTotal() {
        double total = 0;
        for (Produto produto : this.produtos) {
            if (produto != null) {
                total += produto.getPreco();
            }
        }
        return total;
    }
}
```

Solução — o produto mais caro

```java
public Produto getProdutoMaisCaro() {
    Produto maisCaro = null;
    for (Produto produto : this.produtos) {
        if (produto == null) {
            continue;
        }
        if (maisCaro == null || produto.getPreco() > maisCaro.getPreco()) {
            maisCaro = produto;
        }
    }
    return maisCaro;
}
```

- Começar com `null` evita chutar um "preço mínimo" errado
- Estoque vazio devolve `null` — e quem chamar precisa tratar isso

Solução — o item 4 era uma armadilha

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 5 out of bounds for length 5
    at EstoqueDeProdutos.adiciona(EstoqueDeProdutos.java:7)
    at TesteEstoque.main(TesteEstoque.java:15)
```

- Na sexta chamada, `contador` vale 5 — e o array vai de 0 a 4
- É exatamente o problema que deixamos em aberto no `CarrinhoDeCompras`
- Correção possível: testar `if (contador < produtos.length)` antes de escrever

Resumo da aula

- Array guarda vários elementos do mesmo tipo em uma única variável
- É um objeto: precisa de `new` e de um tamanho — que nunca muda
- Índices vão de `0` a `length - 1`; as posições nascem `null` ou zeradas
- `length` é atributo do array; `length()` é método da `String`
- *enhanced-for* para ler, `for` com índice quando o índice importa
- `ArrayIndexOutOfBoundsException`: leia a *stacktrace*, ela dá o endereço
- Tamanho fixo e controle manual são as fronteiras do array

Próxima aula

- **Pacotes, constantes e membros estáticos**
- Nosso projeto vai deixar de ser um monte de arquivos soltos
- Vamos entender de vez os modificadores *default* e `protected`

Atenção: a **Prova 1** é na semana 7.

Tarefa: lista de exercícios da aula 05.